

Programação Orienta a Objetos 1

Encontro 8

Padrões de projeto OO: Singleton, Factory, Builder, Decorator, Proxy, Observer

Objetivos do Encontro

- Apresentar conceitos centrais sobre **Padrões de projeto OO: Singleton, Factory, Builder, Decorator, Proxy, Observer.**
- Discutir exemplos práticos com UML e código.
- Fixar conteúdos com exercícios práticos.

Introdução

Nesta aula abordaremos:

- Definições fundamentais
- Benefícios e aplicações
- Boas práticas

Factory

- **Encapsulamento da criação:** Isola o código de criação de objetos do código que os utiliza
- **Factory Method:** Padrão que cria objetos sem especificar suas classes exatas
- **Abstract Factory:** Fornece interface para criar famílias de objetos relacionados

Factory Method

- **Factory Method:** Padrão que cria objetos sem especificar suas classes exatas

```
class PessoaFactory {  
    public static Pessoa criarPessoa(String nome, int idade) {  
        return new Pessoa(nome, idade);  
    }  
}  
  
public class ExemploFactory {  
    public static void main(String[] args) {  
        Pessoa joao = new Pessoa("João", 30);  
        Pessoa maria = PessoaFactory.criarPessoa("Maria", 25);  
        System.out.println(joao);  
        System.out.println(maria);  
    }  
}
```

Exemplo prático – Factory

```
public class ExemploFactory {  
    public static void main(String[] args) {  
        Date hoje = new Date();  
        Calendar calendario = Calendar.getInstance();  
        //  
        System.out.println("Hoje: " + hoje);  
        System.out.println("Calendário: " + calendario.getTime());  
        calendario.add(Calendar.DAY_OF_MONTH, 5);  
        System.out.println("Daqui a 5 dias: " + calendario.getTime());  
        System.out.println("Intância de Calendar: " + calendario.getClass().getName());  
    }  
}
```

Exemplo prático – Factory

```
abstract class Veiculo {
    abstract void acelerar();
}

class Carro extends Veiculo {
    void acelerar() { System.out.println("Carro acelerando"); }
}

class Moto extends Veiculo {
    void acelerar() { System.out.println("Moto acelerando"); }
}

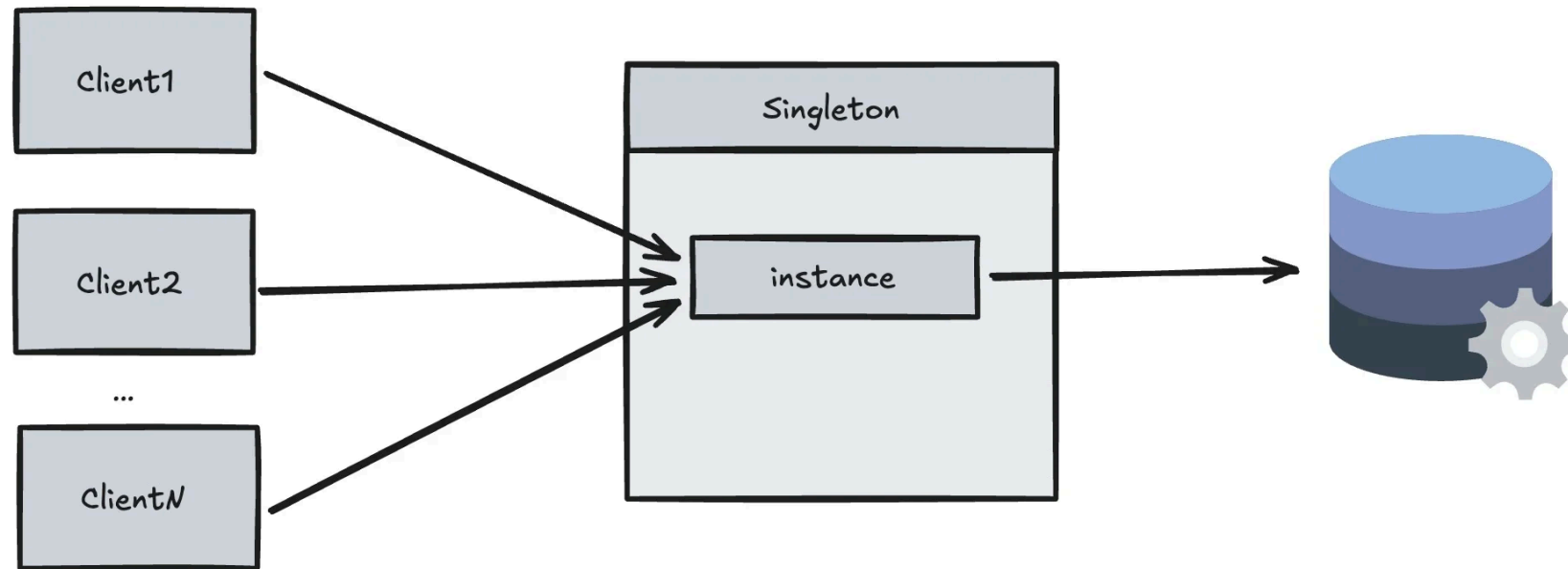
class VeiculoFactory {
    public static Veiculo criarVeiculo(String tipo) {
        if (tipo.equals("carro")) return new Carro();
        if (tipo.equals("moto")) return new Moto();
        return null;
    }
}

// Uso
Veiculo v = VeiculoFactory.criarVeiculo("carro");
v.acelerar();
```

Singleton

- **Singleton:** Padrão que garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a ela.
- **Instância única:** Apenas um objeto da classe pode existir durante a execução do programa.

Singleton



Exemplo prático – Singleton

```
class Configuracao {  
    private static Configuracao instancia;  
    private int alturaJanela;  
    private int larguraJanela;  
    private String tema;  
    //Construtor padrão privado  
    //para evitar instanciamento externo  
    private Configuracao() {  
        tema = "Tema padrão";  
        alturaJanela = 600;  
        larguraJanela = 800;  
    }  
  
    public static Configuracao getInstancia() {  
        if (instancia == null) {  
            instancia = new Configuracao();  
        }  
        return instancia;  
    }  
}
```

Singleton

Inicialização do Singleton:

- **Lazy:** A instância é criada apenas quando necessária.
- **Eager:** A instância é criada no momento do carregamento da classe.

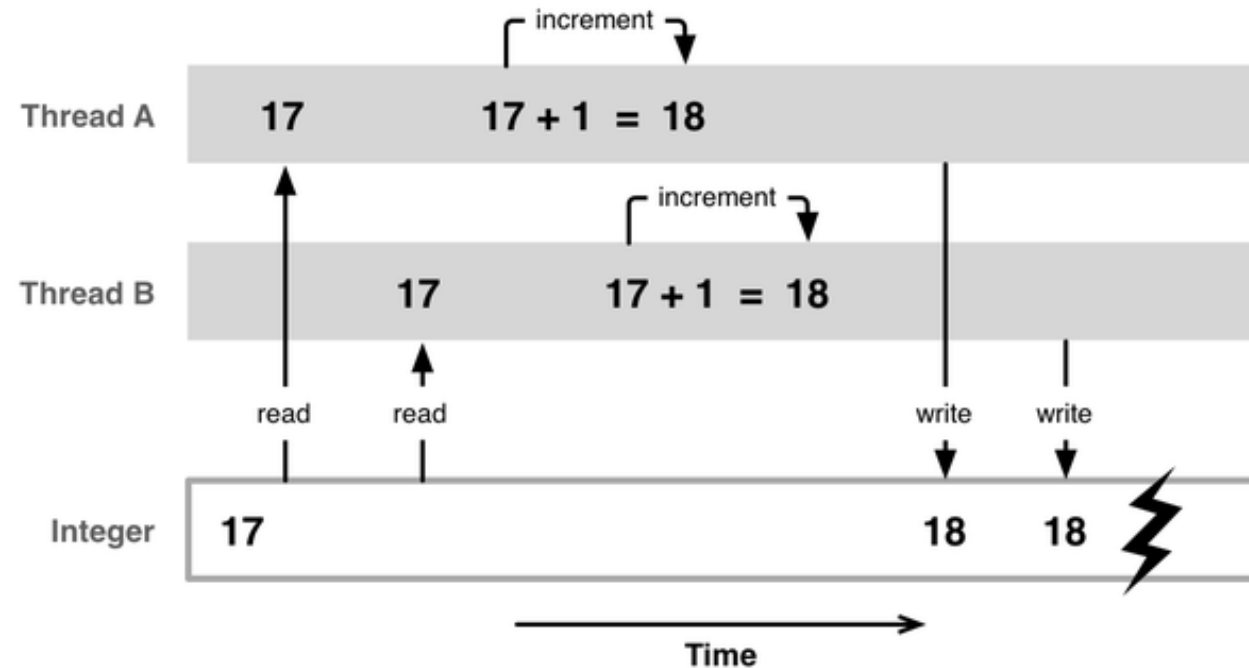
Acesso:

- **Global:** A instância única é acessível de qualquer lugar do programa.
- **Thread safety:** Garantia de que o padrão funcione corretamente em ambientes multi-thread.

Thread Safety em Singleton

Para garantir que o padrão Singleton funcione corretamente em ambientes multi-thread, é importante implementar medidas de segurança. Algumas abordagens incluem:

1. **Sincronização:** Usar blocos sincronizados ou métodos sincronizados para controlar o acesso à instância.



Exemplo condição de corrida

```
public class ExemploCorrida {  
  
    private static int counter = 0; // recurso compartilhado  
  
    public static void main(String[] args) throws InterruptedException {  
  
        Thread[] ts = new Thread[10];  
        for (int i = 0; i < 10; i++) {  
            ts[i] = new Thread(() -> {  
                for(int j = 0 ; j < 1000; j++)  
                    counter = counter + 1;  
  
            });  
        }  
  
        for (Thread t : ts) {  
            t.start();  
        }  
  
        for (Thread t : ts) {  
            t.join();  
        }  
  
        System.out.println("Valor final do contador: " + counter);  
    }  
}
```

Builder

- **Builder:** Padrão que constrói objetos complexos passo a passo.
- **Product:** Objeto complexo sendo construído.
- **Fluent interface:** Interface que permite encadear chamadas de métodos.
- **Separação de construção:** Separa a construção da representação do objeto.
- **Construção incremental:** Permite construir objetos em etapas.

Decorator

- **Decorator**: Padrão que adiciona comportamento a objetos dinamicamente
- **Component**: Interface comum para objetos que podem ser decorados
- **ConcreteComponent**: Implementação básica do Component
- **Decorator**: Classe abstrata que mantém referência ao Component
- **ConcreteDecorator**: Adiciona responsabilidades específicas ao componente
- **Composição**: Usa composição em vez de herança para estender funcionalidade
- **Transparência**: O objeto decorado pode ser usado como o objeto original



Exemplo prático – Decorator

```
interface Cafe {
    double preco();
    String descricao();
}

class CafeSimples implements Cafe {
    public double preco() { return 2.0; }
    public String descricao() { return "Café simples"; }
}

abstract class CafeDecorator implements Cafe {
    protected Cafe cafe;
    public CafeDecorator(Cafe cafe) { this.cafe = cafe; }
}

class ComLeite extends CafeDecorator {
    public ComLeite(Cafe cafe) { super(cafe); }
    public double preco() { return cafe.preco() + 0.5; }
    public String descricao() { return cafe.descricao() + " com leite"; }
}

// Uso
Cafe cafe = new ComLeite(new CafeSimples());
System.out.println(cafe.descricao() + " - R$" + cafe.preco());
```


Exemplo de Decorator

```
import java.io.*;

public class ExemploDecoratorIO {
    public static void main(String[] args) {
        try {
            // InputStream básico (ConcreteComponent)
            InputStream arquivo = new FileInputStream("dados.txt");

            // Decorando com BufferedInputStream (ConcreteDecorator)
            BufferedInputStream bufferizado = new BufferedInputStream(arquivo);
            int bytesDisponiveis = bufferizado.available();

            // Decorando com DataInputStream (ConcreteDecorator)
            DataInputStream dataStream = new DataInputStream(bufferizado);
            dataStream.readInt();

            // Reader básico (ConcreteComponent)
            Reader reader = new InputStreamReader(bufferizado);

            // Decorando com BufferedReader (ConcreteDecorator)
            BufferedReader bufferedReader = new BufferedReader(reader);
            String linha = bufferedReader.readLine();

            System.out.println("Linha lida: " + linha);

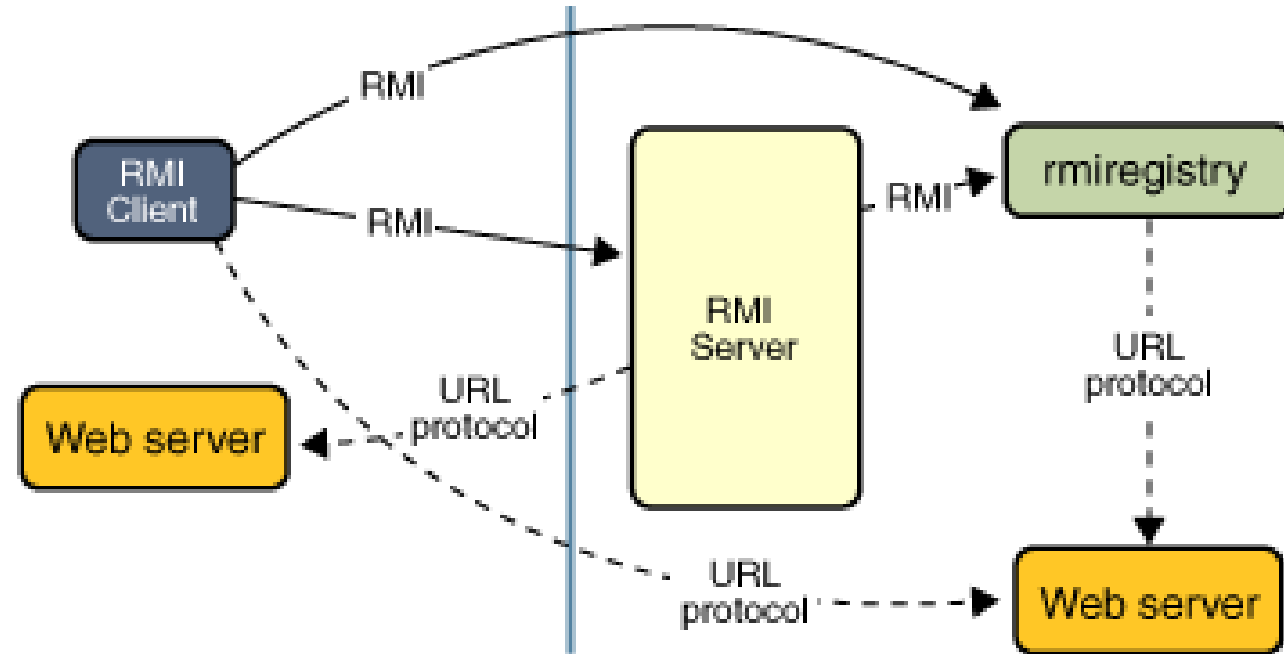
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Proxy

- **Proxy:** Padrão que fornece substituto para outro objeto
- **Subject:** Interface comum para RealSubject e Proxy
- **RealSubject:** Objeto real que o proxy representa

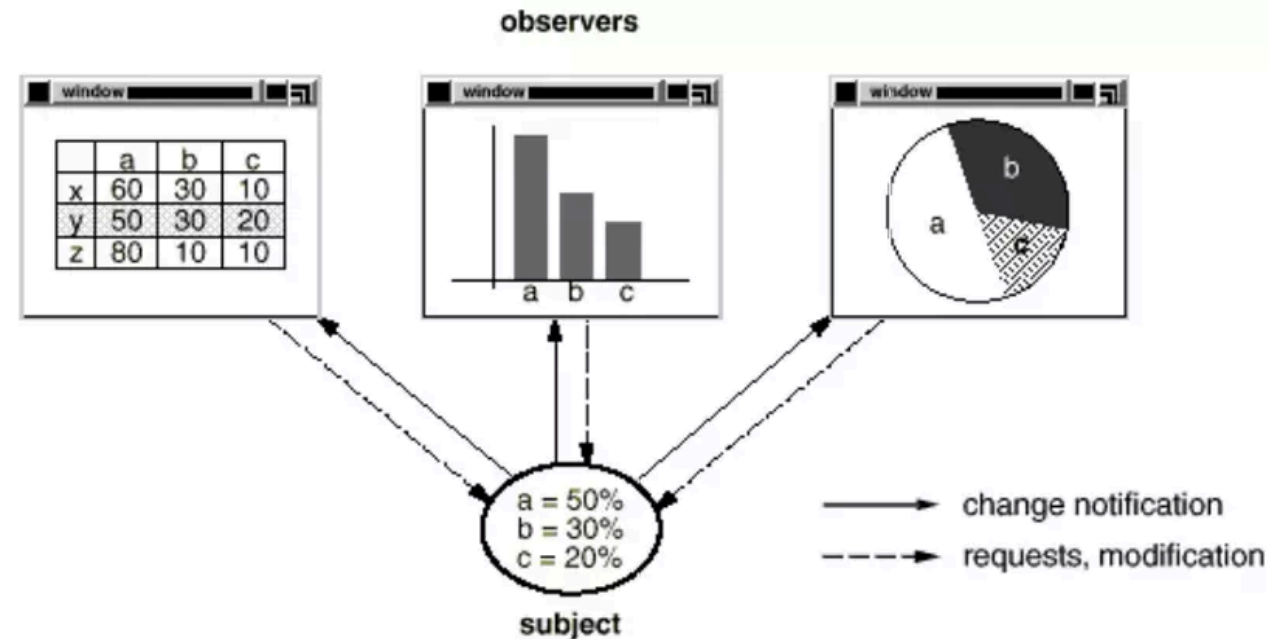
Proxy

- **Proxy:** Mantém referência ao RealSubject e controla acesso a ele
- **Virtual Proxy:** Adia a criação de objetos caros até quando necessário
- **Protection Proxy:** Controla acesso baseado em permissões
- **Remote Proxy:** Representa objeto em espaço de endereço diferente



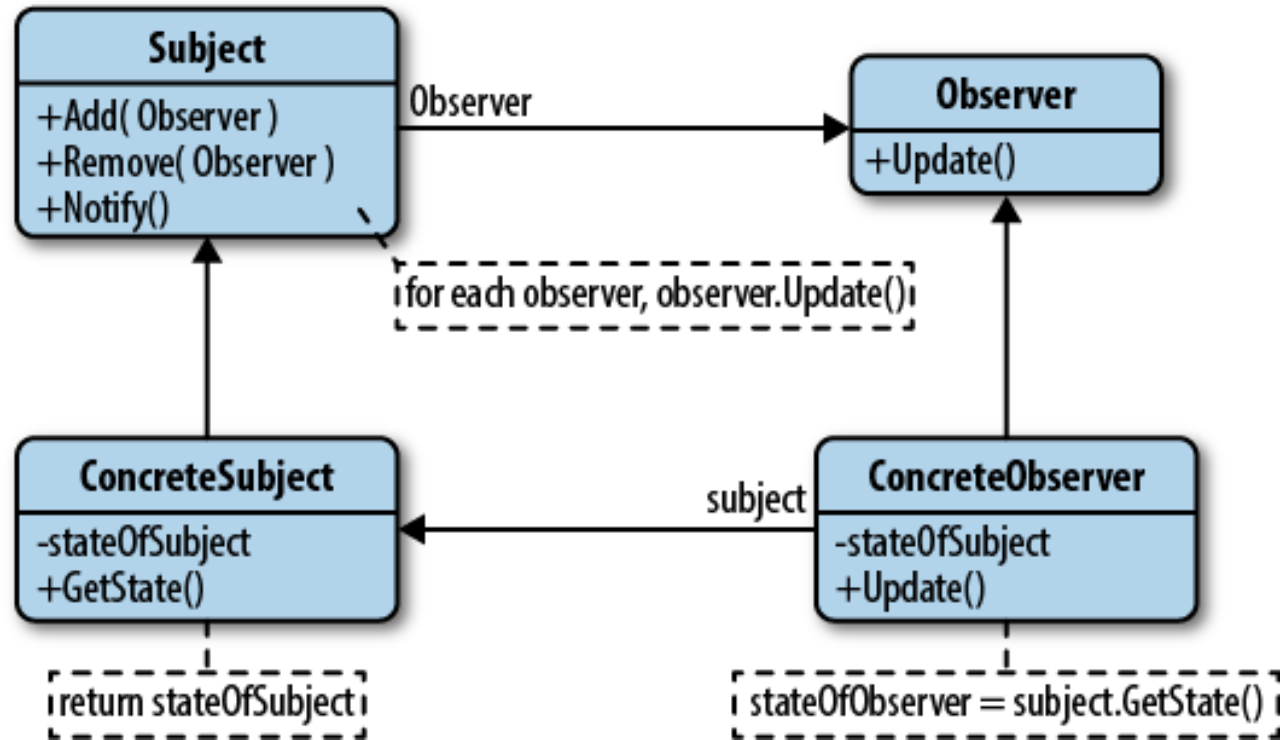
Observer

- **Observer:** Padrão que define dependência um-para-muitos entre objetos
- **Subject/Observable:** Objeto que notifica observadores sobre mudanças de estado
- **Observer:** Interface para objetos que devem ser notificados sobre mudanças



- **ConcreteObserver:**
Implementação específica que reage às notificações
- **ConcreteSubject:**
Implementação que mantém estado e notifica observadores
- **Notificação:** Mecanismo de comunicação entre subject e observers
- **Desacoplamento:**
Observadores e subject são fracamente acoplados
- **Broadcast:** Subject notifica todos os observadores

Observer Pattern



Exemplo prático – Observer

```
import java.util.*;

interface Observer {
    void atualizar(String evento);
}

class Subject {
    private List<Observer> observers = new ArrayList<>();

    public void adicionarObserver(Observer observer) {
        observers.add(observer);
    }

    public void notificar(String evento) {
        for (Observer observer : observers) {
            observer.atualizar(evento);
        }
    }
}
```

```
// Uso
class Cliente implements Observer {
    private String nome;
    public Cliente(String nome) { this.nome = nome; }

    public void atualizar(String evento) {
        System.out.println(nome + " foi notificado: " + evento);
    }
}

Subject loja = new Subject();
loja.adicionarObserver(new Cliente("João"));
loja.adicionarObserver(new Cliente("Maria"));
loja.notificar("Produto em promoção!");
```

Exercício – Observer

No nosso exemplo das aulas anteriores,
transformar nossa classe Produto com o padrão Observer.

De forma que toda alteração de preço deve ser notificado aos interessados que o preço do produto foi alterado.

Referências

- Page-Jones, M. *Fundamentos do Desenho Orientado a Objeto com UML*. Makron Books.
- Larman, C. *Utilizando UML e Padrões*. Saraiva.
- Ementa da disciplina.